NextPlay

Documentación de funciones

219 funciones en 28 archivos · generado el 18/10/2026 20:43

Archivo	Funciones
accounts/admin.py	4
accounts/apps.py	1
accounts/context_processors.py	1
accounts/forms.py	2
accounts/management/commands/nightly.py	1
accounts/management/commands/seed_test_profile.py	3
accounts/management/commands/sync_libraries.py	2
accounts/matching.py	9
accounts/models.py	29
accounts/psn.py	8
accounts/signals.py	1
accounts/steam.py	10
accounts/views.py	29
accounts/xbox.py	9
games/admin.py	1
games/history.py	10
games/igdb.py	15
games/management/commands/import_igdb.py	3
games/management/commands/seed_demo.py	1
games/management/commands/update_prices.py	2
games/models.py	25
games/prices.py	17
games/views.py	4

games/xbox.py	9
library/forms.py	1
library/models.py	8
library/views.py	10
library/wishlist.py	4

App «accounts»

Usuarios: registro, inicio de sesión, perfil público o privado, vinculación con Steam, Xbox y PlayStation y cuentas de otras plataformas.

accounts/admin.py

Clase ExternalTitleAdmin

Revisión de los emparejamientos: los cambiados por usuarios se filtran por «emparejado por».

```
@admin.display(description='usuarios')
def owners(self, obj)
```

Cuántos usuarios tienen este título importado.

accounts/admin.py, línea 61

```
def save_model(self, request, obj, form, change)
```

Si cambia el juego, lo marca como corregido por el administrador y recoloca ya las horas de todos los usuarios que tienen el título (las de un usuario normal esperan a su próxima importación).

accounts/admin.py, línea 65

```
@admin.action(description='Bloquear: los usuarios ya no podrán cambiarlos')
def lock(self, request, queryset)
```

Bloquea los títulos elegidos.

accounts/admin.py, línea 77

```
@admin.action(description='Desbloquear')
def unlock(self, request, queryset)
```

Desbloquea los títulos elegidos.

accounts/admin.py, línea 82

accounts/apps.py

Clase AccountsConfig

```
def ready(self)
```

Registra las señales de la app al arrancar Django (crean el perfil de cada usuario nuevo).

accounts/apps.py, línea 7

accounts/context_processors.py

```
def notifications(request)
```

Número de avisos sin leer del usuario con sesión, para la campana de la cabecera.

accounts/context_processors.py, línea 4

accounts/forms.py

Clase SignupForm

```
def clean_email(self)
```

Pasa el correo a minúsculas y rechaza el registro si ya hay una cuenta con ese correo.

accounts/forms.py, línea 15

Clase PlatformAccountForm

```
def clean(self)
```

Normaliza el nombre o código y comprueba que tiene el formato de la plataforma elegida.

accounts/forms.py, línea 34

accounts/management/commands/nightly.py

Clase Command

```
def handle(self, *args, **options)
```

Lanza cada tarea aunque falle la anterior y da error al final si alguna ha fallado.

accounts/management/commands/nightly.py, línea 14

accounts/management/commands/seed_test_profile.py

Clase Command

```
def add_arguments(self, parser)
```

Opción --vacio para crear la cuenta vacía en vez de la de ejemplo.

accounts/management/commands/seed_test_profile.py, línea 50

```
def handle(self, *args, **options)
```

Crea el usuario de prueba con su biblioteca y reseñas, y añade reseñas de los usuarios demo.

Solo funciona con DJANGO_DEBUG=1. Si el usuario ya existía, rehace su biblioteca desde cero. Con --vacio crea (o deja vacía) la cuenta «tester_steam».

accounts/management/commands/seed_test_profile.py, línea 56

```
def create_empty_user(self)
```

Crea «tester_steam» o lo deja como recién creado: sin juegos ni cuentas de Steam, PlayStation, Xbox u otras plataformas.

accounts/management/commands/seed_test_profile.py, línea 106

accounts/management/commands/sync_libraries.py

Clase Command

```
def add_arguments(self, parser)
```

Opciones: --pause (segundos entre usuarios), --min-hours (saltar a los recién sincronizados) y --user.

accounts/management/commands/sync_libraries.py, línea 24

```
def handle(self, *args, **options)
```

Recorre los usuarios con cuentas de Steam, del que lleva más tiempo sin sincronizar al que menos.

Un usuario que falla (perfil privado, Steam o IGDB caídos) no para a los demás: se apunta en el log y se sigue. Solo da error al final si no ha podido sincronizar a nadie.

accounts/management/commands/sync_libraries.py, línea 36

accounts/matching.py

Emparejamiento de los juegos importados (Steam, PlayStation, Xbox) con el catálogo y su paso a la biblioteca.

Cada título importado se guarda en ExternalTitle (compartido por todos los usuarios) y en ImportedTitle (del usuario, con sus minutos). Los títulos ya emparejados —por una importación anterior o porque alguien lo corrigió a mano— no se vuelven a buscar; los demás se buscan con la función de cada plataforma: Steam por su ID de app y las demás por nombre (en el catálogo local y, si falta, en IGDB). Las horas de cada plataforma en la biblioteca salen después de todos los ImportedTitle del usuario.

```
def clean_name(name)
```

Nombre de un juego sin ™ ® © ni coletillas de plataforma, para buscarlo en el catálogo.

accounts/matching.py, línea 36

```
def normalize(name)
```

Nombre para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

accounts/matching.py, línea 43

```
def add_title(titles, external_id, name, minutes=0, last_played=None, platforms=())
```

Añade un título importado al dict titles ({ID externo: datos}).

Si el ID ya estaba (el mismo juego en dos cuentas de Steam, o dos versiones con el mismo nombre) suma los minutos, se queda con la última partida más reciente y junta las plataformas, que son nombres de plataforma de IGDB («PlayStation 5», «Xbox One»...) para desempatar al buscar por nombre.

accounts/matching.py, línea 48

```
def _platform_names(game)
```

Nombres de las plataformas del juego (de IGDB), aprovechando el prefetch si lo hay.

accounts/matching.py, línea 62

```
def _fits(game, platforms)
```

Indica si el juego puede ser uno jugado en esas plataformas.

Sin plataformas conocidas (del título o del juego) cualquiera vale.

accounts/matching.py, línea 67

```
def match_by_name(titles)
```

Busca en el catálogo los juegos de {clave: (nombre limpio, plataformas)} y devuelve {clave: Game}.

Primero mira el catálogo local; después pide a IGDB por nombre exacto (tal cual y con mayúsculas de título, porque a veces vienen en mayúsculas) los que no tengan ningún candidato de sus plataformas y, para los que sigan sin salir, hace hasta MAX_SEARCHES búsquedas sueltas quedándose solo con resultados de nombre idéntico. Si varios juegos se llaman igual (remakes como God of War o Resident Evil 4) gana el más popular de los que salieron en esas plataformas.

accounts/matching.py, línea 76

```
def save_titles(user, platform, titles, auto_match)
```

Guarda los títulos importados de un usuario en una plataforma y devuelve {ID externo: Game o None}.

titles es el dict que llena add_title. Los títulos que ya tienen juego, o que alguien dejó sin juego a mano, se respetan; los demás se buscan con auto_match(ids), que devuelve {ID: Game}. Los minutos y la última partida del usuario se sobrescriben con los de esta importación.

accounts/matching.py, línea 130

```
def update_library(user, platform, opened_only=False)
```

Pasa a la biblioteca las horas de una plataforma a partir de todos los títulos importados del usuario.

Por juego suma los minutos de sus títulos (puede venir de varias cuentas o de la versión de PS4 y la de PS5) y toma la última partida más reciente. Los juegos nuevos entran como «Jugando» si se jugaron en los últimos días y como «Jugado» si no; sin horas ni partida, como «Pendiente», salvo con opened_only (plataformas que solo listan juegos abiertos alguna vez). Los que ya estaban conservan su estado (salvo «Pendiente», que pasa a jugado si hay horas). Si un título ha cambiado de juego desde la última vez (porque se corrigió el emparejamiento), al juego anterior se le quitan sus horas de esta plataforma. Devuelve (juegos añadidos, juegos actualizados).

accounts/matching.py, línea 180

```
def reassign(imported, game, by)
```

Cambia el juego de un título importado (None = ninguno) para todos y recoloca las horas de quien lo cambia.

Lo hecho por un administrador queda como suyo («admin»); lo de los demás, como «user». Los demás usuarios con ese título lo verán en su próxima importación. Si el juego anterior se queda en la biblioteca de quien lo cambia sin horas, valoración ni reseña, se quita: solo estaba por el error. Lanza PermissionError si el título está bloqueado y quien lo cambia no es administrador.

Devuelve un dict con los juegos añadidos («added») y actualizados («updated») en su biblioteca y si se ha quitado el juego anterior («removed»).

accounts/matching.py, línea 231

accounts/models.py

Clase Profile

```
def __str__(self)
```

Texto del perfil en el admin y en la consola: su nombre visible.

accounts/models.py, línea 33

```
@property
def name(self)
```

Nombre que se muestra: el nombre visible si lo tiene, si no el nombre de usuario.

accounts/models.py, línea 38

```
def get_absolute_url(self)
```

URL del perfil público del usuario (/u/<usuario>/).

accounts/models.py, línea 42

```
def visible_to(self, viewer)
```

Indica si viewer puede ver el perfil: siempre si es público, y si es privado solo su dueño.

accounts/models.py, línea 46

```
@property
def steam_accounts(self)
```

Cuentas de Steam vinculadas, de la primera a la última (aprovecha el prefetch si lo hay).

accounts/models.py, línea 51

```
@property
def steam_linked(self)
```

Indica si el usuario tiene al menos una cuenta de Steam vinculada.

accounts/models.py, línea 56

```
@property
def steam_avatar(self)
```

Avatar de la primera cuenta de Steam vinculada, o cadena vacía si no tiene ninguna.

accounts/models.py, línea 61

```
@property
def psn(self)
```

Cuenta de PlayStation vinculada, o None si no tiene.

accounts/models.py, línea 67

```
@property
def platforms(self)
```

Cuentas de otras plataformas (Xbox, Switch, Epic...) que muestra en su perfil.

accounts/models.py, línea 75

```
@property
def xbox(self)
```

Cuenta de Xbox vinculada, o None si no tiene.

accounts/models.py, línea 80

```
@property
def avatar_url(self)
```

Avatar del perfil: el de Steam, si no el de PlayStation, si no el de Xbox; vacío para usar la inicial.

accounts/models.py, línea 88

Clase SteamAccount

Cuenta de Steam vinculada a un usuario; un usuario puede tener varias.

```
def __str__(self)
```

Texto de la cuenta en el admin: su nombre en Steam, o el SteamID si no se conoce.

accounts/models.py, línea 117

```
@property
def name(self)
```

Nombre que se muestra: el de Steam, o el SteamID si aún no se ha podido leer.

accounts/models.py, línea 122

```
@property
def url(self)
```

URL del perfil en Steam; la construye con el SteamID si Steam no la ha dado.

accounts/models.py, línea 127

Clase PSNAccount

Cuenta de PlayStation Network vinculada a un usuario (una por usuario).

No se guarda ningún código ni token de Sony: el usuario pega su código NPSSO cada vez que sincroniza.

```
def __str__(self)
```

Texto de la cuenta en el admin: su PSN ID, o el ID de cuenta si no se conoce.

accounts/models.py, línea 150

```
@property
def name(self)
```

Nombre que se muestra: el PSN ID, o «PlayStation» si Sony no lo ha devuelto.

accounts/models.py, línea 155

```
@property
def url(self)
```

Perfil público en PSNProfiles (Sony no tiene página pública de perfiles), o vacío sin PSN ID.

accounts/models.py, línea 160

Clase PlatformAccount

Cuenta de otra plataforma que el usuario muestra en su perfil (sin importar juegos).

No se verifica que sea suya: es lo que escribe, como en la mayoría de webs. Una por plataforma.

```
def __str__(self)
```

Texto de la cuenta en el admin: «Plataforma: nombre».

accounts/models.py, línea 215

```
@property
def logo_template(self)
```

Ruta de la plantilla SVG con el logo de la plataforma.

accounts/models.py, línea 220

```
@property
def url(self)
```

Perfil público en la web de la plataforma cuando existe (solo GOG); si no, vacío.

accounts/models.py, línea 225

```
@classmethod
def normalize(cls, platform, handle)
```

Limpia lo que escribe el usuario: quita espacios de los extremos y pone el código de Switch en formato SW-1234-5678-9012 aunque lo escriba solo con números o con espacios.

accounts/models.py, línea 232

```
@classmethod
def is_valid_handle(cls, platform, handle)
```

Indica si el nombre o código cumple el formato de esa plataforma.

accounts/models.py, línea 243

Clase XboxAccount

Cuenta de Xbox (cuenta Microsoft) vinculada a un usuario (una por usuario).

No se guarda ningún token de Microsoft: para sincronizar el usuario vuelve a iniciar sesión.

```
def __str__(self)
```

Texto de la cuenta en el admin: su gamertag.

accounts/models.py, línea 266

```
@property
def name(self)
```

Nombre que se muestra: el gamertag, o «Xbox» si Microsoft no lo ha devuelto.

accounts/models.py, línea 271

```
@property
def url(self)
```

Perfil público del gamertag en xbox.com, o vacío si no se conoce.

accounts/models.py, línea 276

Clase ExternalTitle

Juego tal como lo identifica una plataforma (app de Steam, titleId de PlayStation o de Xbox) y el juego del catálogo que le corresponde (vacío si no se encontró).

Es compartido entre usuarios: la importación lo mira antes de buscar por nombre, así que una corrección hecha a mano sirve para todos. Un administrador puede bloquearlo para que nadie más lo cambie.

```
def __str__(self)
```

Texto del título en el admin: «Plataforma: nombre».

accounts/models.py, línea 325

Clase ImportedTitle

Título que tiene un usuario en una plataforma, con los minutos y la última partida de su última importación.

De aquí salen las horas de cada plataforma en la biblioteca, así que al corregir el juego de un título se pueden recolocar sin volver a importar.

```
def __str__(self)
```

Texto en el admin: «usuario · título».

accounts/models.py, línea 356

```
@property
def hours(self)
```

Horas jugadas redondeadas a una décima.

accounts/models.py, línea 361

Clase Notification

Aviso para un usuario: se ve en la campana de la cabecera hasta que abre la página de avisos.

```
def __str__(self)
```

Texto del aviso en el admin: «usuario · texto».

accounts/models.py, línea 388

accounts/psn.py

Importación de juegos y horas de PlayStation (PS4/PS5) con la API no oficial de la app de PlayStation.

Sony no tiene API pública: se usa la misma que la app oficial, a la que se accede con el código NPSSO de la sesión del usuario en playstation.com. El código no se guarda; se pide en cada sincronización. Es experimental: Sony puede cambiarla sin avisar.

Clase PSNError

Fallo al hablar con PlayStation (red, respuesta inesperada o API cambiada).

Clase PSNAuthError

Sony no acepta el código NPSSO (mal copiado o caducado).

```
def clean_npsso(value)
```

Saca el código NPSSO de lo que pegue el usuario: el código suelto o el JSON entero de la página de Sony.

Devuelve el código o None si no tiene el formato esperado (64 letras y números).

accounts/psn.py, línea 43

```
def access_token(npsso)
```

Cambia el código NPSSO por un token de acceso de la API de PlayStation.

Lanza PSNAuthError si Sony no da el código de autorización (NPSSO incorrecto o caducado).

accounts/psn.py, línea 57

```
def _get(url, token, **params)
```

GET autenticado contra la API de PlayStation; devuelve el JSON.

accounts/psn.py, línea 99

```
def _account_id_from_token(token)
```

ID de cuenta guardado dentro del token JWT, por si el perfil no lo devuelve; None si no está.

accounts/psn.py, línea 111

```
def profile(token)
```

Datos de la cuenta del token: {"account_id", "online_id", "avatar"}.

Si el perfil no responde, intenta sacar al menos el ID de cuenta del propio token.

accounts/psn.py, línea 122

```
def played_games(token)
```

Juegos de PS4 y PS5 que ha jugado la cuenta, con «name», «playDuration», «lastPlayedDateTime»...

accounts/psn.py, línea 142

```
def duration_minutes(value)
```

Minutos de una duración ISO 8601 de PSN («PT228H56M33S» → 13736); 0 si no se entiende.

accounts/psn.py, línea 155

```
def import_library(user, npsso)
```

Vincula (o actualiza) la cuenta de PSN del NPSSO e importa sus juegos de PS4/PS5 con sus horas.

Los nuevos entran como «Jugando» si se jugaron en los últimos 14 días y como «Jugado» si no; los que ya tenía conservan su estado y solo se les actualizan las horas de PlayStation. La versión de PS4 y la de PS5 de un juego se suman. Lanza PSNAuthError si el código no vale y ValueError si esa cuenta de PSN ya es de otro usuario.

Devuelve un dict con la cuenta («account») y cuántos juegos se han añadido («added»), actualizado («updated») y no se han encontrado en el catálogo («unmatched»).

accounts/psn.py, línea 171

accounts/signals.py

```
@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_profile(sender, instance, created, raw=False, **kwargs)
```

Crea el perfil de un usuario justo después de crearlo.

No hace nada al guardar un usuario ya existente ni al cargar fixtures (raw=True).

accounts/signals.py, línea 9

accounts/steam.py

Vinculación con Steam (inicio de sesión OpenID) e importación de los juegos de su biblioteca.

Clase SteamError

Fallo al hablar con Steam (red, clave de la API o respuesta no válida).

Clase SteamPrivateError

Steam no devuelve los juegos porque los detalles del perfil del usuario no son públicos.

```
def is_configured()
```

Indica si está puesta la clave de la Web API de Steam, necesaria para leer la biblioteca.

accounts/steam.py, línea 35

```
def login_url(return_to, realm)
```

URL de Steam donde el usuario inicia sesión; Steam lo devuelve después a return_to.

accounts/steam.py, línea 40

```
def verify_login(params, return_to)
```

Comprueba con Steam la respuesta de OpenID y devuelve el SteamID64 del usuario, o None si no vale.

Rechaza respuestas de otro proveedor, para otra URL de vuelta o que Steam no confirme como suyas (así nadie puede inventarse una URL de vuelta con el SteamID de otro).

accounts/steam.py, línea 53

```
def _api(path, **params)
```

Llama a un método GET de la Web API de Steam con la clave de settings y devuelve su «response».

accounts/steam.py, línea 78

```
def player_summary(steam_id)
```

Nombre, avatar y URL del perfil de Steam: {"persona", "avatar", "profile_url"}, o {} si no lo encuentra.

accounts/steam.py, línea 92

```
def owned_games(steam_id)
```

Juegos de la cuenta de Steam: [{"appid", "name", "playtime_forever", "rtime_last_played"...}].

Lanza SteamPrivateError si el perfil no deja ver sus juegos (Steam responde sin «games»).

accounts/steam.py, línea 105

```
def refresh_account(account)
```

Actualiza el nombre, el avatar y la URL guardados de una cuenta de Steam (sin fallar si Steam no responde).

accounts/steam.py, línea 123

```
def _match_games(app_ids)
```

Busca los juegos del catálogo de una lista de IDs de Steam y devuelve {ID de Steam: Game}.

Primero mira el catálogo local; los que falten los busca en IGDB y los importa (si IGDB está configurado).

accounts/steam.py, línea 135

```
def _played_at(data)
```

Fecha de la última partida de un juego de Steam, o None si nunca se ha jugado.

accounts/steam.py, línea 158

```
def import_library(user, include_unplayed=False)
```

Importa a la biblioteca del usuario los juegos que ha jugado en todas sus cuentas de Steam vinculadas.

Si un juego está en varias cuentas (o en varias apps de Steam, como un relanzamiento) se suman sus horas y se toma la última partida más reciente. Los nuevos entran como «Jugando» si se jugaron en los últimos 14 días y como «Jugado» si no; los que ya tenía conservan su estado y solo se les actualizan las horas. Con include_unplayed también entran los que nunca ha abierto, como «Pendiente». Cada app queda guardada como título importado, así que las que no se encuentren se pueden emparejar a mano. Las cuentas que no dejan ver sus juegos se saltan; si no deja ninguna, lanza SteamPrivateError.

Devuelve un dict con cuántos juegos se han añadido («added»), actualizado («updated») y no se han encontrado en el catálogo («unmatched»), y los nombres de las cuentas privadas («private»).

accounts/steam.py, línea 165

accounts/views.py

```
def signup(request)
```

Registro de cuenta nueva.

Si el usuario ya ha iniciado sesión lo manda a su perfil. Al registrarse bien inicia su sesión y lo lleva al catálogo para que añada sus primeros juegos.

accounts/views.py, línea 42

```
def profile(request, username)
```

Perfil público de un usuario: cabecera con estadísticas, su biblioteca y sus reseñas.

La biblioteca tiene pestañas por estado (Jugando, Completado...) con su contador y se filtra con ?estado=; se ordena con ?orden= (recientes, nota, horas o nombre). Con ?vista=resenas muestra sus reseñas en vez de la biblioteca. Un perfil privado da 404 a cualquiera que no sea su dueño.

accounts/views.py, línea 59

```
def _profile_stats(owner, entries, counts)
```

Cifras de la cabecera del perfil: juegos, completados, horas jugadas (Steam + PlayStation + Xbox), nota media y reseñas.

También calcula los tres géneros más repetidos de su biblioteca.

accounts/views.py, línea 133

```
@login_required
def edit_profile(request)
```

Formulario para que el usuario con sesión cambie su nombre visible, biografía y privacidad.

accounts/views.py, línea 153

```
def linking_context()
```

Variables que necesitan los paneles de vinculación de cuentas (en Editar perfil y en el modal del perfil).

accounts/views.py, línea 167

```
def _back(request, default)
```

Redirige a la página de la que vino el formulario (campo «next») si es de esta web; si no, a default.

Así las acciones lanzadas desde el modal del perfil vuelven al perfil y no a Editar perfil.

accounts/views.py, línea 182

```
def _steam_return_to(request, state)
```

URL absoluta a la que Steam devuelve al usuario tras iniciar sesión, con el código anti-CSRF state.

accounts/views.py, línea 193

```
def _report_import(request, result)
```

Avisa al usuario de cuántos juegos se han importado de Steam, cuántos no estaban en el catálogo y qué cuentas se han saltado por ser privadas.

accounts/views.py, línea 198

```
def _with_unmatched(text, unmatched)
```

Mensaje de resultado de una importación, con el aviso de los juegos no encontrados y un enlace para emparejarlos a mano (HTML seguro: el texto se escapa).

accounts/views.py, línea 212

```
def _import_steam(request, include_unplayed=False)
```

Lanza la importación de todas las cuentas de Steam del usuario y le avisa del resultado o del fallo.

Si falla por Steam o IGDB deja el error completo en el log del servidor para poder revisarlo.

accounts/views.py, línea 223

```
@require_POST
@login_required
def steam_link(request)
```

Manda al usuario a iniciar sesión en Steam para vincular una cuenta más (Steam nunca nos da su contraseña).

No deja pasar de SteamAccount.MAX_PER_USER cuentas.

accounts/views.py, línea 242

```
@login_required
def steam_callback(request)
```

Vuelta desde Steam: verifica el inicio de sesión, añade la cuenta a las del usuario e importa sus juegos.

Rechaza la vuelta si el código anti-CSRF no coincide, si Steam no la confirma o si esa cuenta de Steam ya está vinculada a otro usuario. Si ya era suya, solo avisa.

accounts/views.py, línea 255

```
@require_POST
@login_required
def steam_import(request)
```

Vuelve a importar los juegos de todas las cuentas de Steam vinculadas (con «unplayed» también los no jugados).

accounts/views.py, línea 297

```
@require_POST
@login_required
def steam_auto_sync(request)
```

Activa (auto_sync=1) o desactiva la sincronización nocturna de la biblioteca de Steam del usuario.

accounts/views.py, línea 311

```
@require_POST
@login_required
def steam_unlink(request, account_id)
```

Desvincula una cuenta de Steam del usuario; los juegos importados se quedan en su biblioteca.

Solo puede desvincular cuentas suyas: con el ID de una cuenta ajena responde 404.

accounts/views.py, línea 325

```
def _psn_error(request, text)
```

Avisa del error de PlayStation y vuelve a donde estaba el formulario (por defecto, Editar perfil).

accounts/views.py, línea 338

```
@require_POST
@login_required
def psn_sync(request)
```

Vincula la cuenta de PlayStation del código NPSSO pegado (o la actualiza) e importa sus juegos de PS4/PS5.

El código solo se usa durante esta petición: no se guarda. Avisa si el código no tiene el formato esperado, si Sony no lo acepta o si esa cuenta de PlayStation ya es de otro usuario.

accounts/views.py, línea 346

```
@require_POST
@login_required
def psn_unlink(request)
```

Desvincula la cuenta de PlayStation del usuario; los juegos importados se quedan en su biblioteca.

accounts/views.py, línea 379

```
@require_POST
@login_required
def platform_add(request)
```

Añade la cuenta de otra plataforma (Xbox, Switch, Epic, GOG, Battle.net) o cambia la que ya tenía.

Si el formato no es el de esa plataforma avisa del error y no guarda nada.

accounts/views.py, línea 388

```
@require_POST
@login_required
def platform_remove(request, platform)
```

Quita del perfil del usuario su cuenta de esa plataforma (no da error si no la tenía).

accounts/views.py, línea 409

```
def _xbox_redirect_uri(request)
```

URL absoluta de vuelta desde Microsoft; tiene que coincidir con la registrada en la aplicación de Azure.

accounts/views.py, línea 419

```
@require_POST
@login_required
def xbox_link(request)
```

Manda al usuario a iniciar sesión con su cuenta Microsoft para vincular Xbox o volver a sincronizarla.

Sirve para las dos cosas porque no se guarda ningún token: cada sincronización pide iniciar sesión.

accounts/views.py, línea 426

```
@login_required
def xbox_callback(request)
```

Vuelta desde Microsoft: comprueba el código anti-CSRF, vincula la cuenta de Xbox e importa sus juegos.

Si el usuario canceló el inicio de sesión, si el código no coincide o si Xbox Live falla, avisa y vuelve a Editar perfil.

accounts/views.py, línea 439

```
@require_POST
@login_required
def xbox_unlink(request)
```

Desvincula la cuenta de Xbox del usuario; los juegos importados se quedan en su biblioteca.

accounts/views.py, línea 473

```
def _search_games(query)
```

Juegos del catálogo cuyo nombre contiene query, de más a menos popular, completando con IGDB si salen pocos.

accounts/views.py, línea 486

```
@login_required
def imported_titles(request)
```

Juegos importados para revisar a mano.

Sin parámetros, los títulos de Steam, PlayStation o Xbox del usuario que no se encontraron en el catálogo (con ?ocultos=1, los que ocultó). Con ?juego=<id>, los que han llevado a ese juego de su biblioteca, para corregirlo si no es el que jugó. Con ?titulo=<id>&q= busca candidatos para ese título (por defecto, por su nombre).

accounts/views.py, línea 500

```
@require_POST
@login_required
def imported_title_assign(request, pk)
```

Asigna a un título importado del usuario el juego elegido (o ninguno si «game» llega vacío).

La corrección sirve para todos los usuarios con ese título y sus horas pasan al juego nuevo en su biblioteca. Responde 404 si el título no es suyo o el juego no existe.

accounts/views.py, línea 542

```
@require_POST
@login_required
def imported_title_hide(request, pk)
```

Oculta (hidden=1) un título sin emparejar del usuario —herramientas, demos, apps— o lo vuelve a mostrar.

accounts/views.py, línea 569

```
@login_required
def notifications(request)
```

Últimos avisos del usuario (bajadas de precio...). Al abrir la página, los no leídos pasan a leídos.

accounts/views.py, línea 583

accounts/xbox.py

Vinculación con Xbox (inicio de sesión con cuenta Microsoft) e importación de sus juegos y minutos jugados.

Flujo: el usuario inicia sesión en Microsoft (OAuth 2.0 de una aplicación de Azure con el permiso XboxLive.signin) → con el token de Microsoft se pide uno de usuario de Xbox Live → con ese, uno XSTS que trae el XUID y el gamertag → con el XSTS se leen el historial de juegos y la estadística MinutesPlayed. Ningún token se guarda: se usan durante la petición y se tiran.

Clase XboxError

Fallo al hablar con Microsoft o Xbox Live (red, credenciales o respuesta inesperada).

```
def is_configured()
```

Indica si están puestas las credenciales de la aplicación de Azure para iniciar sesión con Microsoft.

accounts/xbox.py, línea 50

```
def login_url(redirect_uri, state)
```

URL de Microsoft donde el usuario inicia sesión; Microsoft lo devuelve a redirect_uri con un código.

accounts/xbox.py, línea 55

```
def _post(url, **kwargs)
```

POST a Microsoft o Xbox Live que devuelve el JSON; lanza XboxError si falla.

accounts/xbox.py, línea 68

```
def xbox_tokens(code, redirect_uri)
```

Cambia el código de Microsoft por los datos de Xbox Live: {"xuid", "gamertag", "auth"}.

«auth» es la cabecera Authorization (XBL3.0) para llamar a las APIs de Xbox Live.

accounts/xbox.py, línea 88

```
def _get(url, auth, contract, **params)
```

GET autenticado contra una API de Xbox Live con su versión de contrato; devuelve el JSON.

accounts/xbox.py, línea 126

```
def avatar(xuid, auth)
```

URL del avatar (gamerpic) de la cuenta, o vacío si Xbox Live no la da.

accounts/xbox.py, línea 141

```
def played_titles(xuid, auth)
```

Juegos del historial de la cuenta (solo de tipo «Game», sin apps como Netflix), con su última partida.

accounts/xbox.py, línea 151

```
def minutes_played(xuid, auth, title_ids)
```

Minutos jugados por título según la estadística MinutesPlayed: {titleId: minutos}.

Los juegos que no publican esa estadística (muchos de Xbox 360 y algunos de PC) no aparecen.

accounts/xbox.py, línea 157

```
def import_library(user, code, redirect_uri)
```

Vincula (o actualiza) la cuenta de Xbox del inicio de sesión e importa sus juegos con los minutos jugados.

Los juegos se emparejan con el catálogo por nombre (y por los dispositivos en los que se jugaron). Los nuevos entran como «Jugando» si se jugaron en los últimos 14 días y como «Jugado» si no; los que ya tenía conservan su estado (salvo «Pendiente», que pasa a jugado si hay horas) y se les actualizan los minutos de Xbox. Si el usuario tenía puesto a mano un gamertag en «Otras plataformas», se quita porque ya está la cuenta verificada. Lanza ValueError si esa cuenta de Xbox ya es de otro usuario.

Devuelve un dict con la cuenta («account») y cuántos juegos se han añadido («added»), actualizado («updated») y no se han encontrado en el catálogo («unmatched»).

accounts/xbox.py, línea 182

App «games»

Catálogo de juegos, importación desde IGDB y precios de PC (IsThereAnyDeal) y Xbox.

games/admin.py

Clase GamePriceInline

```
def has_add_permission(self, request, obj=None)
```

Impide añadir precios a mano: solo los escribe update_prices.

games/admin.py, línea 13

games/history.py

Historial de precios de PC de un juego (de IsThereAnyDeal), veredicto «¿compro ahora o espero?» y gráfica.

El historial se guarda en Game.price_history como el precio más barato entre todas las tiendas a lo largo del tiempo: un punto [fecha ISO, precio, % de rebaja, tienda] por cada vez que cambia. Se pide a ITAD al abrir la ficha y se reutiliza durante MAX_AGE.

```
def needs_update(game, now=None)
```

Indica si hay que volver a pedir el historial: nunca se pidió o tiene más de MAX_AGE.

games/history.py, línea 36

```
def _amount(value)
```

Importe de un precio de ITAD como Decimal con dos decimales.

games/history.py, línea 42

```
def lowest_series(events, seed, start)
```

Precio más barato entre todas las tiendas a lo largo del tiempo, a partir de los cambios de ITAD.

seed es {id de tienda: (precio, rebaja, nombre)} con lo que cuesta hoy en cada tienda; se usa como precio de partida de las tiendas sin cambios en el periodo. Las que sí tienen cambios empiezan con el precio normal de su primer cambio (antes de una rebaja se vendía a precio normal). Devuelve [[fecha ISO, precio, rebaja, tienda], ...] de más viejo a más nuevo, con un punto solo cuando cambia el precio más barato.

games/history.py, línea 47

```
def update_history(game, client=None, now=None)
```

Pide a ITAD el historial de los dos últimos años del juego y lo guarda. Lanza ITADError si falla.

games/history.py, línea 88

```
def _points(game)
```

Puntos del historial guardado como (fecha, precio Decimal, rebaja, tienda).

games/history.py, línea 106

```
def _year(points, now)
```

Puntos (fecha, precio, rebaja, tienda) del último año, empezando con el que estaba vigente hace un año (con esa fecha) si se sabe.

games/history.py, línea 114

```
def verdict(game, current, now)
```

Veredicto para el precio de hoy: {"code", "title", "text", "year_low"}.

«minimo» si está en su mínimo histórico; «bueno» si en el último año no ha estado más barato o está cerca de lo más barato del año; «espera» si ha bajado de ese precio varias veces.

games/history.py, línea 123

```
def last_big_sale(game)
```

Última vez que estuvo rebajado al menos BIG_SALE_CUT %: {"date", "cut", "price", "shop"}, o None.

games/history.py, línea 153

```
def chart(game, now, current=None, width=320, height=90, pad=6)
```

Datos de la gráfica escalonada del último año para dibujarla en SVG, o None si no hay historial.

Termina en current (el GamePrice más barato de hoy, el que enseña la ficha) si se da: el historial de ITAD puede contar alguna tienda que no sale entre los precios de hoy.

Devuelve el tamaño, los puntos de la línea («line») y del área bajo ella («area»), el precio más alto y más bajo de la gráfica, la altura de la línea del mínimo histórico («low_y», None si queda fuera) y, en JSON («hover»), cada tramo con su fecha, posición, precio, rebaja y tienda para la etiqueta que sale al pasar por encima. Las coordenadas van ya en texto con punto decimal: en una plantilla en español, floatformat pondría coma y el SVG no sería válido.

games/history.py, línea 162

```
def summary(game, now=None)
```

Todo lo que enseña la ficha sobre el historial, o None si no hay historial o precios de PC.

{"current": precio más barato de hoy en PC, "verdict", "chart", "last_big_sale"}.

games/history.py, línea 219

games/igdb.py

Cliente de IGDB y la función que vuelca sus juegos en el catálogo local.

Clase IGDBError

Fallo al hablar con IGDB o con Twitch (credenciales, red o respuesta no válida).

```
def is_configured()
```

Indica si están puestas las credenciales de Twitch para usar IGDB.

games/igdb.py, línea 40

Clase IGDBClient

```
def __init__(self, client_id=None, client_secret=None)
```

Prepara el cliente con las credenciales dadas o las de settings; falla si no hay ninguna.

games/igdb.py, línea 46

```
def _token(self)
```

Token de acceso de Twitch: lo saca de la caché o pide uno nuevo y lo guarda hasta que caduque.

games/igdb.py, línea 53

```
def query(self, endpoint, body)
```

Lanza una consulta en Apicalypse (el lenguaje de IGDB) contra endpoint y devuelve el JSON.

Si IGDB responde 401 borra el token de la caché para que la próxima llamada pida uno nuevo.

games/igdb.py, línea 76

```
def top_games(self, limit=MAX_PAGE_SIZE, offset=0)
```

Una página de los juegos más valorados por número de votos, sin ediciones sueltas, con sus ediciones.

games/igdb.py, línea 99

```
def search(self, term, limit=20)
```

Busca juegos por texto en IGDB (hasta limit resultados, sin ediciones sueltas), con sus ediciones.

games/igdb.py, línea 108

```
def games_by_steam_ids(self, app_ids)
```

Juegos de IGDB (con sus ediciones) de una lista de IDs de Steam y un dict {ID de Steam: id de IGDB}.

Los IDs que IGDB no conoce (herramientas, bandas sonoras, servidores...) no aparecen en el dict.

games/igdb.py, línea 117

```
def games_by_names(self, names)
```

Juegos de IGDB (con sus ediciones) cuyo nombre es exactamente uno de names, sin ediciones sueltas.

Va en lotes para no hacer una petición por juego. Si dos juegos se llaman igual vienen los dos; quien llama decide con cuál quedarse (p. ej. el más popular).

games/igdb.py, línea 142

```
def _with_editions(self, games)
```

Añade a cada juego sus ediciones (GOTY, Definitive...) en la clave «editions», para tener sus IDs de tienda.

games/igdb.py, línea 161

```
def _upsert_ref(model, data, **extra)
```

Busca o crea un Genre o Platform de IGDB, aprovechando uno que no venga de IGDB con el mismo slug.

games/igdb.py, línea 183

```
def _normalize(name)
```

Nombre para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

games/igdb.py, línea 199

```
def _store_ids(data)
```

IDs de tienda de un juego de IGDB: {"steam": [...], "gog": [...]}.

IGDB puede tener varias entradas por tienda: relanzamientos con el mismo nombre que el juego (se quedan todos y sus ofertas se juntan) y otras raras como «Mac Portal» (solo se usa la primera, y únicamente si ninguna se llama como el juego).

games/igdb.py, línea 204

```
def _store_alternatives(data, own_ids)
```

IDs de tienda de las ediciones del juego y después de sus remasters, sin repetir los suyos.

Sirven para encontrar precios cuando el juego original ya no se vende.

games/igdb.py, línea 225

```
def _xbox_product_ids(data)
```

IDs de producto de Microsoft Store del juego y después de sus ediciones y remasters, sin repetidos.

games/igdb.py, línea 244

```
@transaction.atomic
def upsert_games(payload)
```

Crea o actualiza los juegos locales a partir de los de IGDB y devuelve la lista de Game.

Guarda datos, nota, portada, IDs de tienda, géneros y plataformas. Aprovecha juegos de demo o creados a mano con el mismo nombre o slug en vez de duplicarlos. Si cambian los IDs de tienda, borra los precios de PC, que se buscaron con los IDs viejos. Todo va en una transacción.

games/igdb.py, línea 257

games/management/commands/import_igdb.py

Clase Command

```
def add_arguments(self, parser)
```

Opciones del comando: --limit (cuántos populares) y --search (importar una búsqueda).

games/management/commands/import_igdb.py, línea 11

```
def handle(self, *args, **options)
```

Punto de entrada: importa una búsqueda o los más populares y dice cuántos juegos guardó.

games/management/commands/import_igdb.py, línea 16

```
def _import_top(self, client, limit)
```

Importa los limit juegos más populares en páginas de hasta 500. Devuelve cuántos importó.

games/management/commands/import_igdb.py, línea 28

games/management/commands/seed_demo.py

Clase Command

```
def handle(self, *args, **options)
```

Crea los 12 juegos de GAMES con sus géneros y plataformas; se salta los que ya existen.

games/management/commands/seed_demo.py, línea 37

games/management/commands/update_prices.py

Clase Command

```
def add_arguments(self, parser)
```

Opciones del comando: --limit (solo los N más populares) y --game (un solo juego por slug).

games/management/commands/update_prices.py, línea 15

```
def handle(self, *args, **options)
```

Actualiza los precios de PC (si hay ITAD_API_KEY) y los de Xbox de los juegos elegidos, y después crea los avisos de la lista de deseos.

games/management/commands/update_prices.py, línea 20

games/models.py

Clase Genre

```
def __str__(self)
```

Nombre del género.

games/models.py, línea 15

Clase Platform

```
def __str__(self)
```

Nombre completo de la plataforma.

games/models.py, línea 29

```
@property
def short_name(self)
```

Nombre corto para las etiquetas (p. ej. «PS5»); el completo si no tiene abreviatura.

games/models.py, línea 34

Clase Game

```
def __str__(self)
```

Nombre del juego.

games/models.py, línea 96

```
def get_absolute_url(self)
```

URL de la ficha del juego (/juegos/<slug>/).

games/models.py, línea 100

```
@property
def cover_url(self)
```

URL de la portada en el CDN de IGDB, o cadena vacía si el juego no tiene portada.

games/models.py, línea 105

```
@property
def release_year(self)
```

Año de lanzamiento, o None si no se conoce la fecha.

games/models.py, línea 112

```
@property
def price_lookups(self)
```

Grupos ([(tienda, ID en la tienda), ...], nombre de edición) con los que buscar precios, por preferencia.

Primero los IDs del propio juego y después los de sus ediciones y remasters. Los IDs de un mismo grupo son el mismo producto, así que sus ofertas se juntan. Si hay un ID de Steam manual, solo se usa ese.

games/models.py, línea 117

```
@property
def xbox_lookup_ids(self)
```

IDs de producto de Xbox con los que buscar el precio: el manual si lo hay, si no los de IGDB.

games/models.py, línea 135

```
@property
def on_current_xbox(self)
```

Indica si el juego está en Xbox One o Xbox Series X|S.

games/models.py, línea 140

```
@property
def best_price(self)
```

Oferta más barata de todas las tiendas (PC y Xbox), o None si no hay precios.

games/models.py, línea 145

```
@property
def is_free(self)
```

Indica si alguna tienda lo da gratis (precio y precio normal a 0), es decir, si es free-to-play.

games/models.py, línea 151

```
def price_groups(self)
```

Reparte los precios para la ficha del juego en (destacados, listados, plegados, mejor).

destacados: las tiendas más conocidas (Steam, Epic, Microsoft, GOG, Xbox) en un orden fijo. listados: las demás más baratas, solo si el juego no está en ninguna destacada. plegados: el resto por precio, dentro de «Ver N tiendas más». mejor: la oferta más barata cuando ha quedado plegada, para poder señalarla igualmente.

games/models.py, línea 155

```
def store_searches(self)
```

(tienda, plantilla del logo, URL de búsqueda) de las consolas en las que está el juego pero sin precio.

Son PlayStation Store, Nintendo eShop y, si no se encontró precio de Xbox, Xbox Store.

games/models.py, línea 172

```
def community_rating(self)
```

Media de las valoraciones de los usuarios de NextPlay: {"average": 1-5 o None, "count": n}.

games/models.py, línea 185

```
@property
def critic_score(self)
```

Nota de la crítica redondeada (0-100), o None si no tiene.

games/models.py, línea 192

```
@property
def critic_tier(self)
```

Franja de la nota para colorearla: «high» (75 o más), «mid» (50-74), «low» o vacío sin nota.

games/models.py, línea 197

```
@property
def historical_low_symbol(self)
```

Símbolo de la moneda del mínimo histórico (€, $, £...).

games/models.py, línea 205

```
def currency_symbol(code)
```

Símbolo de una moneda a partir de su código ISO («EUR» → «€»); el propio código si no lo conoce.

games/models.py, línea 239

```
def format_price(amount, symbol='€')
```

Precio escrito como en España para los textos que se generan fuera de las plantillas: «2,99 €».

games/models.py, línea 244

Clase GamePrice

Precio actual de un juego en una tienda.

```
def __str__(self)
```

Texto del precio en el admin: «juego · tienda: precio moneda».

games/models.py, línea 272

```
@property
def symbol(self)
```

Símbolo de la moneda del precio.

games/models.py, línea 277

```
@property
def is_free(self)
```

Indica si el juego es gratis en esta tienda (no solo rebajado a 0 por una oferta).

games/models.py, línea 282

```
@property
def is_featured(self)
```

Indica si la tienda es una de las destacadas (Steam, Epic, Microsoft, GOG, Xbox).

games/models.py, línea 287

```
@property
def logo_template(self)
```

Ruta de la plantilla SVG con el logo de la tienda; solo para tiendas destacadas.

games/models.py, línea 292

games/prices.py

Cliente de IsThereAnyDeal y la sincronización que guarda los precios actuales de PC de cada juego.

Clase ITADError

Fallo al hablar con IsThereAnyDeal (clave, red o respuesta no válida).

```
def is_configured()
```

Indica si está puesta la clave de IsThereAnyDeal.

games/prices.py, línea 25

Clase ITADClient

```
def __init__(self, api_key=None, country=None)
```

Prepara el cliente con la clave y el país dados o los de settings; falla si no hay clave.

games/prices.py, línea 31

```
def _post(self, path, body, params=None)
```

POST a la API con la clave y devuelve el JSON.

games/prices.py, línea 38

```
def _get(self, path, params)
```

GET a la API con la clave y devuelve el JSON.

games/prices.py, línea 42

```
def _send(self, method, path, **kwargs)
```

Llama a la API con method (requests.get o requests.post) y la clave, y devuelve el JSON.

Si la API responde 429 (demasiadas peticiones) espera lo que diga Retry-After (máximo 5 minutos) y reintenta hasta 3 veces.

games/prices.py, línea 46

```
def lookup(self, store, store_ids)
```

Traduce IDs de una tienda (de Steam o de GOG) a los UUID de juego de ITAD (None si no lo conoce).

games/prices.py, línea 66

```
def prices(self, itad_ids)
```

Ofertas actuales y mínimo histórico de cada juego de ITAD en el país configurado.

games/prices.py, línea 72

```
def history(self, itad_id, since)
```

Cambios de precio de un juego de ITAD desde la fecha since, en todas las tiendas del país.

Cada cambio es {"timestamp", "shop": {"id", "name"}, "deal": {"price", "regular", "cut"}}; «deal» es None cuando la tienda deja de venderlo. Vienen del más nuevo al más viejo.

games/prices.py, línea 76

```
def _amount(value)
```

Importe de un precio de ITAD como Decimal con dos decimales.

games/prices.py, línea 86

```
def _batches(items)
```

Parte la lista en trozos de BATCH_SIZE, lo máximo que acepta la API por petición.

games/prices.py, línea 91

```
def _cheapest_per_shop(deals)
```

Se queda con la oferta más barata de cada tienda.

games/prices.py, línea 97

```
@transaction.atomic
def _store_prices(game, data, edition='', itad_ids=())
```

Sustituye los precios de PC del juego por las ofertas de data y guarda su mínimo histórico.

Con data vacío deja el juego sin precios de PC. edition es el nombre de la edición de la que salen los precios (vacío si son del propio juego) e itad_ids, los juegos de ITAD de los que salen; si cambian, se borra el historial guardado, que era de otros.

games/prices.py, línea 109

```
def _lookup(client, keys)
```

Traduce un conjunto de (tienda, ID) a UUID de ITAD, tienda por tienda y por lotes.

games/prices.py, línea 145

```
def _fetch_prices(client, itad_ids)
```

Precios de todos los UUID de ITAD pedidos por lotes: {uuid: datos de precios}.

games/prices.py, línea 155

```
def _merge(items)
```

Junta los precios de varios juegos de ITAD que son el mismo producto (p. ej. un relanzamiento).

Suma todas sus ofertas y se queda con el mínimo histórico más bajo. Devuelve None si no hay datos.

games/prices.py, línea 163

```
def update_prices(games, client=None)
```

Actualiza los precios de PC de los juegos y devuelve cuántos acabaron con precios.

Va por rondas: en la primera busca con los IDs del propio juego; los juegos que se quedan sin ofertas pasan a la siguiente ronda con su primera edición o remaster, y así sucesivamente.

games/prices.py, línea 178

```
def lookup_ids(game, client=None)
```

UUID de ITAD de los IDs de tienda con los que se encontraron los precios actuales del juego.

Sirve para los juegos cuyos precios se guardaron antes de que se apuntaran sus itad_ids.

games/prices.py, línea 210

games/views.py

```
def catalog(request)
```

Catálogo de juegos, que también es la portada.

Filtra por nombre (?q=), género (?genero=) y plataforma (?plataforma=) y pagina de 24 en 24. Si hay sesión, marca en cada tarjeta el estado del juego en la biblioteca del usuario. Ofrece importar de IGDB cuando hay búsqueda, sesión y credenciales de IGDB.

games/views.py, línea 21

```
def detail(request, slug)
```

Ficha de un juego: datos, nota de la crítica, valoración de la comunidad, widget de biblioteca, precios y reseñas.

Los precios llegan ya repartidos por Game.price_groups(); store_searches() da los enlaces de búsqueda a las tiendas de consola sin precio. Las reseñas de perfiles privados solo las ve su autor.

games/views.py, línea 66

```
@require_POST
@login_required
def import_search(request)
```

Importa de IGDB los resultados de una búsqueda que no estaban en el catálogo local.

Después vuelve al catálogo con la misma búsqueda y avisa de cuántos juegos se importaron, o de que IGDB no respondió.

games/views.py, línea 106

```
def price_history(request, slug)
```

Historial de precios de PC del juego con el veredicto «¿compro ahora o espero?» (fragmento HTML).

Si el guardado tiene más de un día, lo pide antes a IsThereAnyDeal; si ITAD falla, enseña el que hubiera. La ficha lo carga con HTMX para no esperar a ITAD.

games/views.py, línea 123

games/xbox.py

Precios de consola Xbox sacados del catálogo público de la tienda de Microsoft (sin documentar, sin clave).

Clase XboxError

Fallo al consultar la tienda de Xbox (red o respuesta no válida).

```
def _market()
```

País de la tienda, el mismo que ITAD_COUNTRY (p. ej. «ES»).

games/xbox.py, línea 24

```
def _locale()
```

Idioma y país de la tienda, p. ej. «es-ES».

games/xbox.py, línea 29

```
def fetch_products(product_ids)
```

Devuelve {ID de producto: (precio, precio normal, moneda)} de los productos que se pueden comprar en consola Xbox.

Pide los productos de 20 en 20; los que no se venden en consola no aparecen en el resultado.

games/xbox.py, línea 34

```
def _console_offer(product)
```

Oferta de compra más barata de un producto que funciona en Xbox, o None si no hay.

Ignora las de 0,00, que son Game Pass, pruebas o copias incluidas en packs.

games/xbox.py, línea 59

```
def _normalize(title)
```

Título para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

games/xbox.py, línea 85

```
def search_games(name)
```

IDs de producto de los juegos de Xbox que se llaman exactamente como name (títulos en inglés, como en IGDB).

games/xbox.py, línea 90

```
def store_url(product_id)
```

Enlace a la página del producto en xbox.com.

games/xbox.py, línea 117

```
@transaction.atomic
def _store(game, product_id, offer)
```

Sustituye el precio de Xbox del juego por offer y calcula el % de descuento; sin offer lo deja sin precio.

games/xbox.py, línea 123

```
def update_xbox_prices(games, sleep=time.sleep)
```

Guarda el precio de Xbox de cada juego y devuelve cuántos acabaron con precio.

Usa el primer ID de producto (el suyo y después sus ediciones) que se venda en consola. Como IGDB a menudo solo tiene productos que ya no se venden (p. ej. la versión original de un relanzamiento), los juegos de Xbox One/Series sin precio se buscan también por su título exacto, salvo que otro juego del catálogo se llame igual o que la búsqueda dé más de un producto.

games/xbox.py, línea 141

App «library»

Biblioteca de cada usuario: qué juegos tiene, en qué estado, su valoración y su reseña.

library/forms.py

Clase ReviewForm

```
def clean_review(self)
```

Quita los espacios del principio y del final para que una reseña en blanco cuente como vacía.

library/forms.py, línea 18

library/models.py

Clase LibraryEntryQuerySet

```
def reviews_visible_to(self, viewer)
```

Entradas con reseña que viewer puede leer, de la más reciente a la más antigua.

Las de perfiles privados solo las ve su autor, igual que su biblioteca.

library/models.py, línea 16

Clase LibraryEntry

```
def __str__(self)
```

Texto de la entrada en el admin: «usuario · juego · estado».

library/models.py, línea 81

```
@classmethod
def imported_status(cls, minutes, last_played)
```

Estado de un juego que llega de una importación (Steam, PlayStation o Xbox) según cuánto y cuándo se jugó.

Sin minutos ni partida: «Pendiente». Jugado en los últimos IMPORT_RECENT_DAYS días: «Jugando». Si no: «Jugado».

library/models.py, línea 86

```
def update_import(self, platform, minutes, last_played)
```

Guarda los minutos y la última partida de una plataforma ("steam", "psn" o "xbox") en una entrada existente.

Conserva el estado que puso el usuario, salvo «Pendiente» cuando ya hay horas jugadas: entonces lo cambia a «Jugando» o «Jugado». Devuelve True si ha cambiado algo.

library/models.py, línea 98

```
@property
def playtime_minutes(self)
```

Minutos jugados sumando Steam, PlayStation y Xbox, o None si no viene de ninguna.

library/models.py, línea 119

```
@property
def playtime_hours(self)
```

Horas jugadas (Steam + PlayStation + Xbox) redondeadas a una décima, o None si no hay datos.

library/models.py, línea 127

```
@property
def steam_hours(self)
```

Horas jugadas en Steam redondeadas a una décima, o None si no viene de Steam.

library/models.py, línea 133

Clase WishlistItem

Juego que el usuario quiere («Lo quiero»), con un precio por debajo del cual avisarle (opcional).

Los avisos los crea library.wishlist.check_alerts después de actualizar los precios; también avisa cuando el juego está rebajado a su mínimo histórico, aunque no haya precio objetivo.

```
def __str__(self)
```

Texto en el admin: «usuario · juego».

library/models.py, línea 164

library/views.py

```
def review_context(game, entry, viewer)
```

Variables de la sección de reseñas de un juego: las reseñas que viewer puede leer, cuántas son y el formulario de su propia reseña (None si el juego no está en su biblioteca).

library/views.py, línea 19

```
def _respond(request, game, entry)
```

Respuesta común de las acciones de biblioteca.

Si la petición viene de HTMX devuelve el widget de biblioteca ya actualizado junto con el recuadro de valoración de la comunidad y la sección de reseñas (que HTMX cambia en su sitio); si no, redirige a la ficha del juego.

library/views.py, línea 30

```
@require_POST
@login_required
def set_status(request, game_id)
```

Añade el juego a la biblioteca del usuario con el estado enviado, o cambia el que tenía.

Responde 400 si el estado no es uno de los permitidos.

library/views.py, línea 57

```
@require_POST
@login_required
def remove(request, game_id)
```

Quita el juego de la biblioteca del usuario (no da error si no estaba).

library/views.py, línea 74

```
@require_POST
@login_required
def rate(request, game_id)
```

Guarda la valoración del usuario (1 a 5 estrellas) para un juego de su biblioteca.

Con rating=0 borra la valoración. Responde 404 si el juego no está en su biblioteca y 400 si la valoración no es un número del 0 al 5.

library/views.py, línea 83

```
@require_POST
@login_required
def review(request, game_id)
```

Guarda, cambia o borra (si llega vacía) la reseña del usuario sobre un juego de su biblioteca.

Responde 404 si el juego no está en su biblioteca. Si la reseña no es válida (demasiado larga) avisa del error. Siempre vuelve a la sección de reseñas de la ficha del juego.

library/views.py, línea 104

```
def _wish_response(request, game, wish, error='')
```

Respuesta de las acciones de la lista de deseos: el botón «Lo quiero» actualizado si es HTMX; si no, vuelve a la página de la que vino (la ficha por defecto) avisando del error si lo hay.

library/views.py, línea 130

```
@require_POST
@login_required
def wish_toggle(request, game_id)
```

Añade el juego a la lista de deseos del usuario o lo quita si ya estaba.

library/views.py, línea 145

```
@require_POST
@login_required
def wish_target(request, game_id)
```

Guarda el precio por debajo del cual avisar (vacío = solo avisar del mínimo histórico).

Al cambiarlo se olvida el último aviso, para volver a comparar con el precio nuevo. Responde 404 si el juego no está en su lista.

library/views.py, línea 157

```
@login_required
def wishlist(request)
```

Lista de deseos del usuario con el mejor precio de hoy de cada juego, su mínimo histórico y si está por debajo de su límite. Se ordena con ?orden= (recientes por defecto, descuento, precio o nombre).

library/views.py, línea 187

library/wishlist.py

Avisos de la lista de deseos: comparan los precios de hoy con lo que quiere cada usuario.

Se lanzan al terminar update_prices. Un juego avisa cuando su mejor precio (PC o Xbox) baja del precio que puso el usuario o cuando está rebajado a su mínimo histórico. No se repite el aviso hasta que baje más, o hasta que vuelva a subir y baje otra vez. Los juegos que ya están en la biblioteca del usuario no avisan (ya lo tiene).

```
def alert_reason(item, best)
```

Por qué avisar del precio best (un GamePrice) de un juego de la lista: «objetivo», «minimo» o None.

«minimo» solo cuenta si está rebajado: un juego que nunca baja siempre está en su mínimo.

library/wishlist.py, línea 24

```
def alert_text(item, best, reason)
```

Texto del aviso: el juego, el precio, la tienda y el motivo.

library/wishlist.py, línea 39

```
def check_alerts(games=None)
```

Crea los avisos de bajada de precio de las listas de deseos (de esos juegos, o de todos) y manda un correo por usuario con los suyos si lo tiene activado. Devuelve cuántos avisos ha creado.

library/wishlist.py, línea 47

```
def _send_email(user, notifications)
```

Manda al usuario un correo con sus avisos de precio; si falla, lo apunta en el log y sigue.

library/wishlist.py, línea 83